AI-Enhanced Honeypots: Leveraging LLM for Adaptive Cybersecurity Responses

Jason Aljenova Christli
Information Technology Department
Swiss German University
Tangerang, Banten, Indonesia
jason.christli@student.sgu.ac.id

Charles Lim
Information Technology Department
Swiss German University
Tangerang, Banten, Indonesia
charles.lim@sgu.ac.id

Yevonnael Andrew
Information Technology Department
Swiss German University
Tangerang, Banten, Indonesia
yevonnael.andrew@sgu.ac.id

2024 16th International Conference on Information Technology and Electrical Engineering (ICITEE) | 979-8-3503-7581-7/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICITEE62483.2024.10808265

***Abstract*—Honeypots have long been used as decoy systems to lure and study attackers, providing valuable intelligence on emerging cybersecurity threats. However, with the increasing sophistication of cyberattacks, static honeypots have become less effective against advanced adversaries. This paper presents an innovative solution: AI-driven honeypots powered by Large Language Models (LLMs), specifically the LLaMA-3 model. Unlike traditional honeypots, these AI-enhanced systems can dynamically generate contextually appropriate, human-like responses in real-time, greatly improving their ability to deceive and engage attackers. By leveraging the LLaMA-3 model, the proposed system enhances the realism of interactions, making it significantly more difficult for attackers to identify the decoy.**

***Index Terms*—Cybersecurity, Honeypots, Large Language Models, LLaMA-3, Adaptive Systems, SSH**

I. Introduction

Honeypots are decoy systems designed to lure and observe attackers, offering detailed insights into their techniques and tactics [1]. By mimicking real systems, honeypots play a pivotal role in cybersecurity, providing valuable intelligence that informs and strengthens defense mechanisms. Over time, these systems have evolved from simple traps into sophisticated tools that contribute significantly to cybersecurity strategies [2].

However, as cyber threats grow more sophisticated, the need for more advanced defensive measures has become critical [3]. Traditional honeypots, which often rely on static and predictable responses, struggle to deceive and engage skilled attackers effectively [4]. Recent research highlights the importance of enhancing honeypots to better capture and analyze attacker behaviors. For example, studies on behavior similarity detection in Linux shell honeypots demonstrate that frameworks like MITRE ATT&CK can improve the attribution and analysis of attacker activity [5]. Additionally, applying natural language processing (NLP) techniques to map Linux shell commands to MITRE ATT&CK tactics has shown that understanding command execution patterns is crucial for identifying sophisticated attack strategies and increasing honeypot effectiveness [6].

One of the most common services targeted by attackers is Secure Shell (SSH), a critical protocol used for secure remote access and system management [7]. Due to its widespread use, SSH has become a prime target for cyberattacks [8]. While traditional security mechanisms such as firewalls and intrusion detection systems provide some defense, they often fall short against more advanced threats. SSH honeypots are designed to attract attackers exploiting SSH vulnerabilities, offering valuable intelligence on their methods [8]. Yet, like other traditional honeypots, SSH honeypots often suffer from static responses, which makes them less effective at deceiving skilled adversaries [9].

In response to these limitations, advancements in Artificial Intelligence (AI) offer promising solutions. AI has become a powerful tool in cybersecurity [10], with applications ranging from anomaly detection and threat prediction to automating incident responses [11]. AI models, particularly large language models (LLMs), have shown great potential for processing large datasets, recognizing patterns, and generating human-like responses [12]. Recent advancements in transformer architectures have significantly improved the capabilities of LLMs [13], making them more effective for dynamic, contextually relevant tasks.

One such model, LLaMA-3 [14], a large language model text-generation AI developed by Meta, presents a valuable opportunity for enhancing honeypot systems. When integrated into honeypots, particularly in SSH environments, LLaMA-3, also knows as Big Language Model Meta AI 3, can generate realistic, contextually appropriate responses to attacker queries, making the honeypot appear more legitimate. This dynamic interaction improves the honeypot's ability to deceive attackers and gather intelligence, addressing the limitations of static response systems [15].

Following are the key contributions of this research:

- Design of dynamic honeypots utilizing AI models for high realism.
- Integration of LLMs to enhance adaptability to attacker behaviors.
- Evaluation of system realism using Levenshtein distance.
- Recommendations for improving honeypot command coverage and persistent memory.

This research aims to design and implement advanced honeypot systems that leverage AI models to simulate real network environments with a higher level of realism. By integrating the LLaMA-3 model's AI capabilities, the study seeks to enable dynamic, intelligent responses that can adapt

Authorized licensed use limited to: Univ of New Haven. Downloaded on March 08,2025 at 00:13:51 UTC from IEEE Xplore. Restrictions apply.

to the evolving tactics of cyber attackers. The AI-driven honeypot developed in this research will analyze, learn from, and respond adaptively to specific attacker queries and commands, thereby enhancing its ability to deceive adversaries and improving its overall operational effectiveness.

The remainder of this paper is organized as follows: In section II, we review related work on honeypot technologies, the role of AI in cybersecurity, and the integration of LLMs in honeypot systems. Section III outlines the research framework, detailing the development and testing of the AI-enhanced honeypot. In Sections IV and V, we present the experimental setup and discuss the results, respectively. Finally, section VI concludes our research and future works.

II. Related Works

A. Honeypot Technologies

Honeypots are decoy systems designed to mimic real network environments to attract and study cyber attackers. These systems play a crucial role in cybersecurity by providing valuable insights into attacker behaviors and tactics. Honeypots come in various types and interaction levels, including low-interaction honeypots that simulate basic services and capture limited attack patterns, medium-interaction honeypots that simulate more complex services while operating within a controlled environment, and high-interaction honeypots that provide attackers with a real operating system to interact with. High-interaction honeypots, though more complex to deploy and manage, offer the most detailed insights into sophisticated attack strategies by capturing comprehensive data on attacker activities and techniques [1].

B. Limitations of Traditional Honeypots

Despite their utility, traditional honeypots face several limitations. Their static nature makes them easily detectable by skilled attackers, reducing their effectiveness in capturing sophisticated attack techniques. Fan et al. [16] noted that traditional honeynets lack the capability of dynamically adapting their configuration after deployment, which makes them less effective in evolving attack scenarios. Touch and Colin [17] further highlighted that these systems often provide limited engagement and fail to adapt to evolving attack strategies, underscoring the need for more dynamic and adaptive honeypot systems to address these challenges.

C. Artificial Intelligence in Cybersecurity

Artificial Intelligence (AI) refers to the simulation of human intelligence in machines that are programmed to think and learn like humans. AI has significantly advanced various domains, including cybersecurity, by offering robust capabilities for threat detection, response, and analysis. AI techniques such as machine learning and natural language processing (NLP) are used to analyze vast amounts of data, identify patterns, and automate responses to cyber incidents. Prasad et al. [10] highlighted the role of AI in enhancing cybersecurity measures, particularly through AI models that can detect anomalies and predict potential threats, thus improving the overall security posture of systems.

Holzinger et al. [11] discussed the potential of AI in achieving human-level concept learning, which is critical for understanding complex cyber threats and devising appropriate countermeasures. Their research emphasized the importance of explainable AI (XAI) in cybersecurity, ensuring that AI-driven decisions are transparent and interpretable, which is crucial for trust and accountability in cybersecurity applications. These advancements in AI provide powerful tools for developing more adaptive and effective cybersecurity defenses, including AI-enhanced honeypots.

D. Large Language Models

Table I describes the comparison of leading LLMs, demonstrating LLaMA-3's advantages in resource efficiency and overall performance. while Table II outlines the licensing details, showcasing the costeffectiveness of LLaMA-3 as an open-source model

Large Language Models (LLMs) are advanced AI systems designed for natural language processing tasks such as translation, summarization, question-answering, and content generation. LLMs like GPT-4, Claude, and LLaMA-3 are valuable in cybersecurity for automating threat detection, analyzing data, and providing insights into vulnerabilities [18] [19]. In cybersecurity, LLMs enhance digital security by detecting anomalies, predicting threats, and automating incident responses, thereby improving overall system security [18] [19].

TABLE I
Performance and Specification Comparison of LLMs

Model	Parameters	Context Window	MMLU 5-shot
GPT-4 Turbo	175B	128k tokens	86.4%
GPT-4	175B	8k tokens	86.4%
GPT-3.5 Turbo	175B	16k tokens	82.0%
Claude 3 Opus	200B	200k tokens	88.2%
Claude 3 Sonnet	110B	200k tokens	81.5%
Claude 3 Haiku	60B	200k tokens	75.0%
Llama 3 70B	70B	8k tokens	82.0%
Llama 3 8B	8B	8k tokens	68.4%
Code Llama	70B	8k tokens	81.7%
Mistral Large	13B	8k tokens	80.0%
Mistral Medium	7B	8k tokens	78.0%
Mistral Small	3B	8k tokens	75.0%
Mixtral 8x22B	22B	8k tokens	79.0%
Mixtral 8x7B	7B	8k tokens	77.0%
Mistral 7B	7B	8k tokens	78.0%
DBRX	80B	8k tokens	85.0%

LLaMA-3 is the most suitable large language model for integration into honeypot systems due to the following key factors:

1) **Resource Efficiency:** LLaMA-3 8B, with a smaller parameter size (8B) compared to larger models like GPT-4 (175B), requires fewer computational resources. This makes it feasible for systems with limited hardware, without sacrificing performance.
2) **Cost-Effectiveness:** As an open-source model under the Meta LLaMA-3 Community License, LLaMA-3 is free to use, eliminating the licensing costs associated with proprietary models such as GPT-4. This makes it an economical choice, particularly for long-term projects with budget constraints.

Authorized licensed use limited to: Univ of New Haven. Downloaded on March 08,2025 at 00:13:51 UTC from IEEE Xplore. Restrictions apply.

3) **Cutting-Edge Technology:** Released in April 2024, LLaMA-3 incorporates the latest advancements in AI research, offering state-of-the-art performance for natural language processing tasks.
4) **Open-Source Flexibility:** The open-source nature of LLaMA-3 allows researchers and developers to easily modify and improve the model to meet specific requirements, enhancing its adaptability in different use cases.
5) **Advanced Contextual Understanding:** Despite its smaller parameter count, LLaMA-3 8B demonstrates strong language comprehension and produces highly accurate, contextually relevant responses, making it ideal for creating realistic and engaging interactions in honeypot systems.

TABLE II
COMPARISON OF LICENSING INFORMATION FOR VARIOUS LLM MODELS

Model	License	Release Date
GPT-4 Turbo	Proprietary (OpenAI)	Nov 2023
GPT-4	Proprietary (OpenAI)	Mar 2023
GPT-3.5 Turbo	Proprietary (OpenAI)	Nov 2023
Claude 3 Opus	Proprietary (Anthropic)	Mar 2024
Claude 3 Sonnet	Proprietary (Anthropic)	Mar 2024
Claude 3 Haiku	Proprietary (Anthropic)	Mar 2024
Llama 3 70B	Meta Llama 3 Community License	Apr 2024
Llama 3 8B	Meta Llama 3 Community License	Apr 2024
Code Llama	Meta Llama 3 Community License	Apr 2024
Mistral Large	Apache License 2.0	Sep 2023
Mistral Medium	Apache License 2.0	Sep 2023
Mistral Small	Apache License 2.0	Sep 2023
Mixtral 8x22B	Apache License 2.0	Sep 2023
Mixtral 8x7B	Apache License 2.0	Sep 2023
Mistral 7B	Apache License 2.0	Sep 2023
DBRX	Proprietary	Feb 2024

E. AI-Enhanced Honeypots

Integrating Artificial Intelligence (AI) into honeypot systems significantly advances cybersecurity by enhancing threat detection, response, and analysis capabilities. AI-enhanced honeypots, such as QRASSH by Pauna et al. [20], use reinforcement learning to dynamically interact with attackers and gather valuable threat intelligence. These systems are more adaptive and harder to identify, continuously improving their effectiveness. AI models can analyze vast amounts of data, identify patterns, and generate human-like responses, making them suitable for enhancing honeypot interactions. Recent advancements in AI, such as the development of large language models (LLMs) like GPT-3 and LLAMA-3, have demonstrated significant potential in generating contextually appropriate and realistic responses, making them valuable for enhancing cybersecurity measures [16].

F. Integration of LLMs in Honeypot Systems

AI-driven honeypots dynamically generate responses based on real-time analysis of attacker behavior, enhancing their deception capabilities. Ragsdale and Boppana [21] developed low-risk honeypots using generative pre-trained transformers to generate realistic responses, improving deception effectiveness. The LLAMA-3 model, with its advanced natural language processing capabilities, is particularly well-suited for this purpose, providing highly realistic and contextually relevant responses [15]. This integration aims to overcome the limitations of traditional honeypots and provide a more robust and effective tool for cybersecurity defenses.

In summary, while significant advancements have been made in enhancing honeypot systems through the use of AI and machine learning models, many approaches still face limitations in generating adaptive, contextually accurate responses. This research builds on these prior efforts by integrating the LLaMA-3 model, an advanced LLM, to generate dynamic, real-time responses in a honeypot environment. By addressing the limitations of static and predictable honeypots, this study aims to improve the deception capabilities of honeypots, making them more resistant to detection by skilled attackers.

III. RESEARCH FRAMEWORK

A. Overview

This section outlines the methodology adopted to develop and test the AI-enhanced honeypot system, focusing on SSH environments. The approach involves two primary tasks: the development of the honeypot using the SDLC Waterfall Model and the testing of the honeypot using specific command similarity metrics.

Figure 1 illustrates the overall research framework flowchart, which follows a sequential process for developing and testing the honeypot.

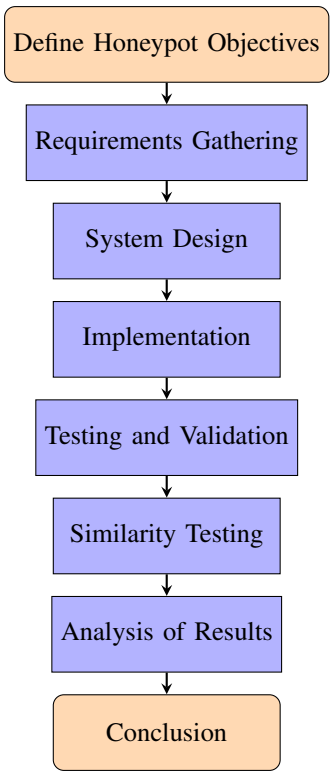

Fig. 1. Research Framework Flowchart

Authorized licensed use limited to: Univ of New Haven. Downloaded on March 08,2025 at 00:13:51 UTC from IEEE Xplore. Restrictions apply.

B. Honeypot Development Using the SDLC Waterfall Model

The development process follows the SDLC Waterfall Model [22], which begins with requirements gathering to define goals and identify constraints. In the system design phase, the architecture integrates the LLaMA-3 model with the SSH honeypot, as shown in Figure 2. The LLaMA-3 model, accessed via a Huggingface API, generates real-time responses to SSH commands, enhancing deception by simulating realistic, adaptive system behavior. The system processes attacker inputs via Paramiko-based SSH, returning contextually generated responses.

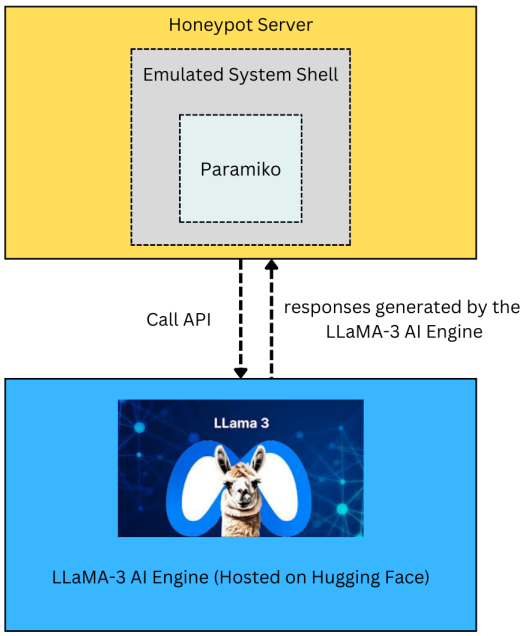

Fig. 2. Overall System Architecture

In the implementation phase, SSH server configurations using Paramiko handle connections and commands [23], while API calls to the LLaMA-3 model generate dynamic responses. A logging system is developed to capture and store interactions for analysis. The testing and validation phase ensures the system meets all requirements and performs as expected.

Figure 3 presents a detailed workflow of the honeypot system's operation. On the left, the figure outlines the initialization process, which involves starting the system, importing necessary libraries, authenticating with HuggingFace, loading the LLaMA-3 model, setting up logging, and configuring the SSH server. Once these tasks are completed, the system enters a listening mode, awaiting incoming connections.

The right side of Figure 3 illustrates the steps each thread follows for every incoming connection. Once a connection is established, the system checks for timeout or exit conditions. If the connection remains active, command inputs are processed and sent to the AI engine via an API call. The AI engine then generates a contextually appropriate response, which is returned to the attacker to simulate a realistic interaction. All interactions are logged for further analysis,

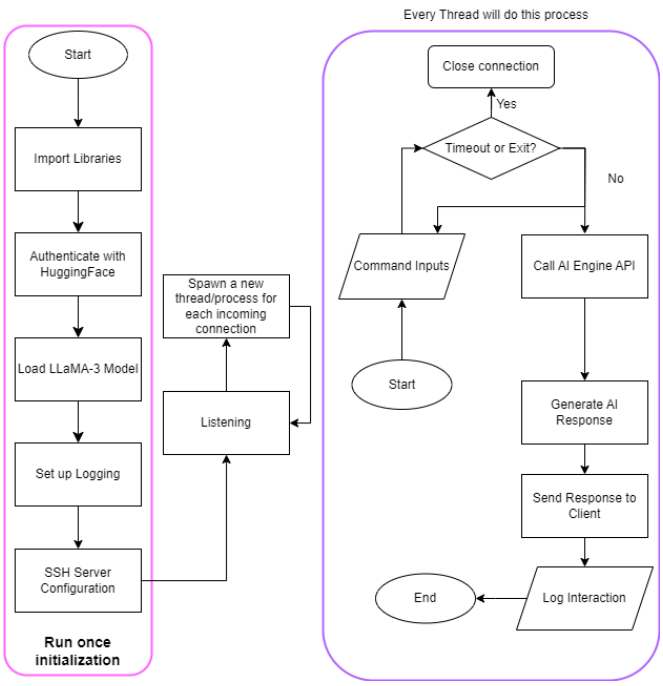

Fig. 3. Operational Workflow of the Honeypot System

ensuring that every exchange is managed dynamically to enhance the honeypot's deception capabilities.

C. Honeypot Testing Using Command Similarity Metrics

The testing phase evaluates the honeypot's ability to generate realistic responses and its overall effectiveness. Commands are categorized into system commands, filesystem commands, and connectivity commands [21]. A control environment is established to provide a baseline for comparing the honeypot's responses against real system responses.

Data collection involves capturing all interactions between the honeypot and attackers, with logs stored securely for subsequent analysis. The Levenshtein Distance metric is employed to measure the similarity between honeypot responses and real system responses, inspired by the approach detailed by Ragsdale and Boppana in their work on low-risk honeypots using generative pre-trained transformer models [21]. Evaluation criteria are defined to assess the realism and effectiveness of the responses.

The evaluation phase focuses on analyzing the similarity scores to determine how closely the honeypot responses mimic real system behavior [21]. Additionally, the honeypot's effectiveness is evaluated based on its ability to engage attackers and gather valuable intelligence [21].

D. Levenshtein Distance Metric

Levenshtein distance measures sequence differences based on the minimum number of character edits. [24]. Equation 1

Authorized licensed use limited to: Univ of New Haven. Downloaded on March 08,2025 at 00:13:51 UTC from IEEE Xplore. Restrictions apply.

shows the formula for Levenshtein distance measurement:

$$d_{i,j} = \begin{cases} \max(i,j) & \text{if } \min(i,j) = 0, \\ \min \begin{cases} d_{i-1,j} + 1, \\ d_{i,j-1} + 1, \\ d_{i-1,j-1} + [a_i \neq b_j] \end{cases} & \text{otherwise.} \end{cases} \tag{1}$$

where $d_{i,j}$ is the distance between the first i characters of one string and the first j characters of the other [24]. Levenshtein distance was chosen for this study due to its ability to measure the similarity between two sequences by calculating the minimal number of single-character edits required. This makes it particularly suitable for comparing system command outputs in text-based environments, where subtle differences in output can be crucial in detecting deviations between real system behavior and honeypot responses.

IV. Experiment Setup

The experiment was conducted using a carefully selected combination of hardware and software to ensure the successful development and testing of the AI-driven honeypot system.

The hardware included an Intel Xeon Gold 5217 CPU running at 3.00GHz with 16 cores, providing ample processing power, along with 62GB of RAM and a 195GB SSD for data storage (with 16GB available during the experiment). The system was connected via a high-speed Ethernet connection. The VMware SVGA II Adapter was used as the graphics processor, and the system was running on Ubuntu 22.04.4 LTS.

On the software side, the experiment was carried out using Python 3.8.10 as the primary programming language. Several key libraries were utilized, including Paramiko [23] for managing SSH sessions and the `Requests` library for API integration with Hugging Face, where the LLaMA-3 model was hosted. The `Subprocess` module facilitated the execution of shell commands within the emulated system environment, while Python's logging module was essential for capturing and storing detailed interaction logs.

V. Research Results

A. Honeypot Functionality Testing

Functionality testing confirmed the honeypot's ability to initiate and execute various SSH commands, demonstrating its operational effectiveness. The system successfully set up and generated responses from the AI model, accurately emulating SSH shell commands like `ls`, `pwd`, and `whoami`. Additionally, all command executions and responses were properly logged, providing valuable data for further analysis and insight into system performance.

The functionality tests affirm that the AI-driven honeypot effectively mimics an SSH environment, engages attackers with realistic interactions, and logs all activities for subsequent analysis. These results establish a foundation for further evaluation and enhancement.

B. Evaluation Metrics and Similarity Calculations

1) System Commands: System commands were executed in both the honeypot and a control environment, with the similarity of responses measured using the Levenshtein distance. The results, summarized in Table III, indicate that commands such as `uname -a` exhibit high similarity, reflecting the honeypot's effective emulation of these system functions. However, commands like `dmesg` highlight areas where the emulation is less accurate, pointing to specific opportunities for further refinement in the system's response capabilities.

TABLE III
Comparison of System Command Outputs

Command	L-ratio	Similarity
uname -a	0.747477945	High
dmesg	0.707577994	Low
ps aux	0.016156403	Low
top	0.005091018	Low
lscpu	0.247379745	Low
lsmem	0.302453526	Low
lsblk	0.306103497	Low
cat /proc/cpuinfo	0.032698427	Low
cat /proc/meminfo	0.363503244	Moderate

2) Filesystem Commands: The honeypot's performance in handling filesystem commands, as detailed in Table IV, shows strong results. High similarity scores for commands such as `pwd` and `chown` indicate the system's robust capability to accurately emulate filesystem interactions, enhancing its effectiveness in deceiving attackers.

TABLE IV
Comparison of Filesystem Command Outputs

Command	L-ratio	Similarity
ls -l	0.771812081	High
pwd	1.0	High
df -h	0.358446344	Moderate
du -sh	0.364185703	Moderate
find / -name file1	0.316157523	Low
cat /etc/passwd	0.136126644	Low
chmod 644 file1	0.365790365	Moderate
chown user:group file1	1.0	High
mkdir folder	0.300300303	Low
rm -rf directory_or_file	0.425539191	Low

3) Connectivity Commands: Table V summarizes the evaluation of network-related operations. The command `ping -c 4 8.8.8.8` shows high similarity, demonstrating the honeypot's effectiveness in handling network diagnostics.

TABLE V
Comparison of Connectivity Command Outputs

Command	L-ratio	Similarity
ifconfig	0.255102041	Low
ping -c 4 8.8.8.8	0.907905135	High
netstat -tuln	0.552374052	Moderate

C. Analysis of Results

The AI-driven honeypot has demonstrated significant effectiveness in emulating real system responses, particularly

Authorized licensed use limited to: Univ of New Haven. Downloaded on March 08,2025 at 00:13:51 UTC from IEEE Xplore. Restrictions apply.

excelling in commands related to system and connectivity, such as `uname -a` and `ping`, which show high similarity to genuine responses. However, areas needing improvement have been identified, with low similarity scores in commands like `dmesg` and `lsblk` , indicating that the honeypot's interaction capabilities in these cases require further refinement. These findings suggest that future development should focus on improving the system's response accuracy for such commands, expanding the AI model's coverage to handle a broader range of interactions. Additionally, incorporating persistent memory would allow the system to maintain session states, further enhancing its deception capabilities and realism.

D. Limitations

The current implementation of the AI-driven honeypot system has several limitations that future research should address to enhance its effectiveness. One key limitation is the lack of persistent memory, which restricts the system's ability to retain state information across sessions, reducing the continuity of interactions. Additionally, the command coverage is incomplete, with certain commands not being accurately emulated, which can undermine the realism of the honeypot. Another challenge is the limited dynamic response generation, where the system's adaptability to diverse attacker behaviors is not fully developed. Resource constraints, particularly limited hardware capabilities, also impact overall performance. Finally, the testing environment's controlled nature may not fully replicate real-world conditions, affecting the generalizability of the results.

VI. Conclusion

The integration of AI, particularly the LLaMA-3 model, into honeypot systems marks a significant advancement in cybersecurity. The AI-driven honeypot developed in this research has demonstrated a high degree of realism in mimicking real system responses, crucial for deceiving and engaging sophisticated attackers. Leveraging LLM enables the honeypot to generate contextually relevant, human-like responses in real-time, making it a more effective tool for gathering intelligence on attacker behavior. This research contributes to the evolving field of adaptive cybersecurity by bridging the gap between traditional static honeypots and modern AI-enhanced solutions. Our future work would focus on improving AI-driven honeypots by implementing persistent memory, expanding command set support, and fine-tuning AI models for more accurate responses. In addition, real-world evaluations and exploring other AI models could further enhance their effectiveness against evolving cyber threats.

References

[1] L. Spitzner, *Honeypots: Tracking Hackers*. Addison-Wesley, 2002.

[2] V. A. Memos and K. E. Psannis, "Ai-powered honeypots for enhanced iot botnet detection," in *2020 3rd World Symposium on Communication Engineering (WSCE)*, 2020, pp. 64–68.

[3] I. H. Sarker, M. H. Furhad, and R. Nowrozy, "Ai-driven cybersecurity: an overview, security intelligence modeling and research directions," *SN Computer Science*, vol. 2, no. 3, p. 173, 2021.

[4] N. Ilg, P. Duplys, D. Sisejkovic, and M. Menth, "Survey of contemporary open-source honeypots, frameworks, and tools," *Journal of Network and Computer Applications*, vol. 103737, 2023.

[5] K. Hobert, C. Lim, and E. Budiarto, "Semantic similarities for honeypot collected linux shell commands," in *Proceedings of the 9th International Conference on Information and Communication Technology (ICoICT)*, 2023, pp. 1–6.

[6] Y. Andrew, C. Lim, and E. Budiarto, "Mapping linux shell commands to mitre att&ck using nlp-based approach," in *Proceedings of the 9th International Conference on Information and Communication Technology (ICoICT)*, 2023, pp. 1–6.

[7] T. Ylonen, "Rfc 4251: The secure shell (ssh) protocol architecture," 2006, rFC.

[8] C. Hetzler, Z. Chen, and T. M. Khan, "Analysis of ssh honeypot effectiveness," in *Future of Information and Communication Conference*. Springer Nature Switzerland, March 2023, pp. 759–782.

[9] B. B. Gupta and A. Gupta, "Assessment of honeypots: Issues, challenges and future directions," *International Journal of Cloud Applications and Computing (IJCAC)*, vol. 8, no. 1, pp. 21–54, 2018.

[10] R. Prasad and V. Rohokale, "Artificial intelligence and machine learning in cyber security," in *Cyber security: the lifeline of information and communication technology*, 2020, pp. 231–247.

[11] A. Holzinger, A. Saranti, A. Angerschmid, B. Finzel, U. Schmid, and H. Mueller, "Toward human-level concept learning: Pattern benchmarking for ai algorithms," *Patterns*, 2023.

[12] S. Wang, T. Hu, H. Xiao, Y. Li, C. Zhang, H. Ning, R. Zhu, Z. Li, and X. Ye, "Gpt, large language models (llms) and generative artificial intelligence (gai) models in geospatial science: a systematic review," *International Journal of Digital Earth*, vol. 17, no. 1, p. 2353122, 2024.

[13] A. R. Sajun, I. Zualkernan, and D. Sankalpa, "A historical survey of advances in transformer architectures," *Applied Sciences*, vol. 14, no. 10, p. 4316, 2024.

[14] Meta, "Llama 3 model," https://www.llama.com/docs/model-cards-and-prompt-formats/other-models#meta-llama-3, April 2024, (Accessed on 09/30/2024).

[15] W. Huang, X. Zheng, X. Ma, H. Qin, C. Lv, H. Chen, J. Luo, X. Qi, X. Liu, and M. Magno, "An empirical study of llama3 quantization: From llms to mllms," 2024, arXiv.

[16] W. Fan, D. Fernández, and Z. Du, "Adaptive and flexible virtual honeynet," in *Mobile, Secure, and Programmable Networking: First International Conference, MSPN 2015, Paris, France, June 15-17, 2015, Selected Papers 1*. Springer International Publishing, 2015, pp. 1–17.

[17] S. Touch and J.-N. Colin, "A comparison of an adaptive self-guarded honeypot with conventional honeypots," *Applied Sciences*, vol. 12, no. 10, p. 5224, 2022.

[18] NVIDIA Developer Blog, "Bolstering cybersecurity: How large language models and generative ai are transforming digital security," 2024, accessed: July 30, 2024.

[19] C. Labs, "Applying ai - llms in cybersecurity," 2024, accessed: July 30, 2024. [Online]. Available: https://www.cirruslabs.io/hubfs/Applying%20AI%20-%20LLMs%20in%20Cybersecurity.pdf

[20] A. Pauna, A.-C. Iacob, and I. Bica, "Qrassh-a self-adaptive ssh honeypot driven by q-learning," in *2018 International Conference on Communications (COMM)*, 2018, pp. 441–446.

[21] J. Ragsdale and R. V. Boppana, "On designing low-risk honeypots using generative pre-trained transformer models with curated inputs," *IEEE Access*, vol. 11, pp. 117 528–117 545, 2023.

[22] Teaching Agile, "Waterfall model of sdlc," 2024, accessed: July 30, 2024. [Online]. Available: https://teachingagile.com/sdlc/models/waterfall

[23] Paramiko, "Paramiko - sshv2 protocol library," 2024, accessed: July 30, 2024. [Online]. Available: https://www.paramiko.org/

[24] B. Berger, M. S. Waterman, and Y. W. Yu, "Levenshtein distance, sequence comparison and biological database search," *IEEE Transactions on Information Theory*, vol. 67, no. 6, pp. 3287–3294, May 2020.

Authorized licensed use limited to: Univ of New Haven. Downloaded on March 08,2025 at 00:13:51 UTC from IEEE Xplore. Restrictions apply.